An Efficient Parallel Algorithm for Linear Programming Problems

Ming-Chang Lee

Department of Information Management, Fooyin University,

Tai-Liao Hsiang Kaohsiung Hssien, Taiwan, ROC

Sc114@mail.fy,edu.tw

Abstract

This study developed a parallel algorithm to efficiently solve linear programming models. The proposed algorithm utilizes the Dantzig-Wolfe Decomposition Principle and can be easily implemented in a general distributed computing environment. The analytical performance of the new algorithm, including the speedup upper bound and lower bound limits, was derived. Numerical experiments are also provided in order to verify the complexity of the proposed algorithm. The empirical results demonstrate that the speedup of this parallel algorithm approaches linearity, which means that it can take full advantage of the distributed computing power as the size of the problem increases.

Keywords: linear programming, parallel algorithm, speedup

1. Introduction

Linear programming (LP) involves a sequence of steps that will lead to the most effective way to allocate scarce resources among competing activities. LP is widely used in a number of areas to help managers make decisions, such as assigning jobs to machines, mixing ingredients for a product, determining a distribution system and other situations. An LP model consists of an objective function to be optimized and mathematical statements of the constraints. Given that many LP models represent large and complex physical systems, a typical medium-sized LP model might have 20,000 variables and 5,000 constraints [1]. The required computing resources for solving a modest LP application are therefore huge.

The availability of cost-effective parallel computers has shown the potential of distributed computing power for many large-scale mathematical programming problems. Some previous studies have developed interesting results in this area (e.g., [2,3]). Robert et al. [4] use a branch- and – bound approach in solving mixed integer programming problems, which use MIP package software (MPSX v2). When the branch – and -

bound search is underway there will be many active nodes, each one of which involves the solution of an LP. These LPs are independent and can be solved in parallel if multiple processors are available. Viswanathkumar and Srinivasan [5] used a branch and bound algorithm to minimize completion time variance on a single processor.

However, exploiting parallelism with a mathematical programming algorithm is not always easy due to the communication complexity between processors, which often becomes a bottleneck during the execution process. Despite many software tools developed for the distributed computing environment, to convert a conventional application into a parallel application remains very difficult. For instance, many Operations Research textbooks (Hiller, Lieberman, and Lieberman [6]) have introduced the simplex method, which is an algebraic procedure for solving linear programming problems. The simplex method improves the feasible solution in an orderly manner by performing a series of elementary row operations until the optimality is achieved. To execute the simplex method in a parallel mode is apparently difficult due to its sequential nature.

In this study, we developed a parallel algorithm, based on the Dantzig-Wolfe decomposition principle (DWDP), to solve linear programming and other block-type optimization problems [7]. Although Dantzig and Wolfe introduced the decomposition principle in the early sixties, it is still widely adopted to cope with large-scale optimization problems. Given that larger and more complex mathematical models have become commonplace (Chinneck [8]), the importance of the DWDP is well recognized by researchers. Using both analytical studies and numerical analysis, we show that the proposed parallel algorithm can be executed efficiently in a general distributed computing environment.

This paper is organized as follows. The next section develops the parallel linear programming model. We explore the analytical performance of the proposed algorithm in Section 3. Computational results are shown in section 4 where the new algorithm is implemented in a portable parallel-computing environment. Finally, concluding remarks are presented in Section 5.

2 Description of the algorithm

Consider a linear programming problem that can be expressed in the following form.

$$\text{Minimize} \quad \mathbf{c}^T\mathbf{x} \qquad (1)$$

$$\text{Subject to:} \quad \mathbf{A}\mathbf{x} = \mathbf{b}$$

$$\mathbf{x} \geqq \mathbf{0}$$

where $\mathbf{A}$ is a matrix of order m by k, $\mathbf{c}$ and $\mathbf{x}$ are both k-dimensional vectors and $\mathbf{b}$ is an m-dimensional vector with each component nonnegative. It is

observed that the **A** matrix in many large linear programming problems usually has a special block-angular structure, namely:

$$\mathbf{A} = \begin{bmatrix} \mathbf{L_1} & \mathbf{L_2} & \cdots & \mathbf{L_n} \\ \mathbf{A_1} & & & \\ & \mathbf{A_2} & & \\ & & \ddots & \\ & & & \mathbf{A_n} \end{bmatrix} \quad (2)$$

$$\mathbf{x_i} \geq \mathbf{0}$$

Where all $\boldsymbol{A_i}$ in the technology matrix $\boldsymbol{A}$ are independent blocks linked by coupling-equation matrices $\boldsymbol{L_i}$. As the angular-structure appears, the decomposition principle is substantiated by forming an equivalent master program (defined below) and several subproblems which correspond to each sub-matrix $\boldsymbol{A_i}$. The solution procedure for (1) involves iterations between a set of independent subproblems where their objective functions are formed using parameters derived from the master program.

Suppose each $\boldsymbol{A_i}$ has m_i rows and k_i columns and each $\boldsymbol{L_i}$ is an $m_0 \times k_i$ matrix, for i = 1, 2, …, n. By partitioning vectors **b**, **x** and **c** into sizes corresponding to each $\boldsymbol{A_i}$, problem (1) can be rewritten as follows.

Minimize

$$\sum_{i=1}^{n} \mathbf{c}_i^T \mathbf{x}_i \quad (2)$$

Subject to: $\sum_{i=1}^{n} \mathbf{L}_i \mathbf{x}_i = \mathbf{b}_0$

$$\mathbf{x_i} \in \boldsymbol{\Omega}_i$$

where $\boldsymbol{\Omega}_i = \{ \boldsymbol{A_i x_i} = \mathbf{b_i}, \mathbf{x_i} \geq \mathbf{0} \}$ for i = 1, 2, …, n. Apparently, the set $\boldsymbol{\Omega}_i$ is convex and mutually independent.

We can then define the subproblem i, for i = 1, 2, …, n, as:

Minimize $(\mathbf{c}_i^T - \mathbf{1_0}^T \boldsymbol{L}_i)\, \mathbf{x}_i$ (3)

Subject to: $\boldsymbol{A_i}\, \mathbf{x_i} = \mathbf{b_i}$,

where $\mathbf{1_0}^T$ is the vector denoting the simplex multipliers corresponding to the constraint $\sum_{i=1}^{n} \mathbf{L}_i \mathbf{x}_i = \mathbf{b}_0$.

In contrast to the subproblem (3), problem (2) is called the master program. Based on the property of convexity of (2) and (3), which implies that all solutions can be written as a linear combination of their vertices, a two-level algorithm for the solution of the linear programming problem can be developed. The master program is on the first level in searching for the coefficients of the linear combination and the subproblem (3) is on the second level for solving the possible optimal vertices. Details of this two-level algorithm, which applies the Dantzig-Wolfe decomposition principle, can be found in Martinson and Tind [9].

Assume that there exists a distributed computing environment (DCE) in which the processing units are independent machines, connected by a

network, and a centralized processor (or the master processor) serves as the coordinator. Such an environment has proven to be a viable approach to provide concurrent computing power at reasonable costs [10]. The design of an algorithm on DCE requires tight load balancing in order to reduce the communication overhead and obtain good performance [11]. A parallel two-level algorithm for linear programming problems that can be implemented in a general DCE is described below.

Algorithm 1. Parallel LP method

Step 1 Initiate the distributed computing environment by creating n processes in the network and assign one of these processes as the master process to coordinate the computing tasks.

Step 2 Let basis matrix $\boldsymbol{B} = \boldsymbol{I}$, be an identity matrix.

Step 3 The master process solves the current basic solution $\boldsymbol{X_B}$, and finds the simplex multipliers $1^T = \boldsymbol{B^{-1}c_B}^T$ where $1^T = (1_0^T, \bar{l})$ and $\bar{l} = (\bar{l}_1, \bar{l}_2, \ldots, \bar{l}_n)$.

Step 4 The master process broadcasts necessary data to each child-process and assigns the ith child-process to solve the ith subproblem (as denoted in equation (3)) where each child-process calculates

$$r_i^* = (\boldsymbol{c}_i^T - 1_0^T \boldsymbol{L}_i)\mathbf{x}_i^* - \bar{l}_i$$

for $i = 1, 2, \ldots, n$.

Step 5 Once the ith child-process solves the ith subproblem, it sends r_i^* and $\mathbf{x}_i^*$ to the master process. After all of the processes return their solutions, if all $r_i^* \geq 0$, then the algorithm terminates. Otherwise, go to Step 6.

Step 6 The master process determines which column the basis is entered by selecting the minimum value r_i^* of the subproblems. Let $\begin{pmatrix} \mathbf{L}_i \mathbf{x}_i^* \\ \mathbf{e}_i \end{pmatrix}$ be the column that will enter the current basis $\boldsymbol{B}$, where $\boldsymbol{e}_i$ is a unit vector.

Step 7 The master process updates $\boldsymbol{B^{-1}}$ and go to Step 3.

In the presented algorithm, Step 1

declares the distributed computing environment and creates n child processes. Step 2 assigns the initial basic feasible solution for the master problem. Each child-process uses the simplex multipliers $1^{\mathbf{T}}$, found in Step 3, to solve the ith subproblem in Step 4. Note that this step is the most time consuming part in a sequential algorithm because n linear programming models must be solved. The master process collects the solutions obtained from each subproblem, determines the optimal solution $\mathbf{x}_i^*$ and the associated optimal objective value r_i^*, and checks the optimal condition in Step 5. If the condition is satisfied, $\mathbf{x}_i^*$ is the extreme point of $\mathbf{\Omega}_i$ and the optimum is found. Step 6 constructs the corresponding vector that will enter the basis of the master program if the terminating condition is not satisfied. The solutions $\mathbf{x}_i^*$ for the ith subproblems are then sent to the master program, which combines these inputs to update the basic solution matrix and determines a new 1^{T}. The result is again sent to each child-process and the iteration proceeds until an optimality test is satisfied.

3 Analytical performance

To evaluate the performance of the proposed algorithm stated in the previous section, first we will investigate to performance complexity. The presented algorithm could be implemented into a general distributed computing environment consisting of a network of heterogeneous computers.

Assume that the parallel algorithm uses a cluster of p workstations connected in a DCE and terminated in time T_p. Let T_s be the best possible time required to solve the same problem using a sequential (uni-processor) algorithm. The ratio

$$S_p = T_s / T_p \qquad (4)$$

is called the algorithm speedup [12]. Speedup is one of the most common indicators for measuring the efficiency of a parallel algorithm. For simplicity, assume that there are enough processors to execute the n subproblem in parallel. If is the execution time of the inherently sequential proportion of the algorithm, and is the remaining proportion that could be performed in parallel (the execution time is /n for a system with n processor) then $T_s = t(\alpha + \beta)$, and $T_p = t(\alpha + \frac{b}{n})$. Therefore, the speedup could be approximated by

$$S_p \cong \frac{a+b}{a+\frac{b}{n}} \qquad (5)$$

When is a proportion of , that is, $= w$, then

$$S \equiv \frac{nw+n}{nw+1} \qquad (6)$$

We can further derive the speedup upper

格式化

格式化

格式化

bound and lower bound limits as follows.

Upper bound of $S_p = \lim_{w \to 0} S_p = n$ (7)

Lower bound of $S_p = \lim_{w \to \infty} S_p = 1$ (8)

Now, when the number of subproblems, n, is greater than the number of processors, p, that are available, and assume that $n = (p\text{-}1)\,q$, the speedup could be approximated by:

$$S_p \cong \frac{nw+n}{nw+1} = \frac{(p-1)qw+(p-1)q}{(p-1)qw+1} < \frac{(p-1)qw+(p-1)q}{(p-1)qw+q}$$

$$= \frac{(p-1)w+(p-1)}{(p-1)w+1} \quad (9)$$

The speedup upper bound and lower bound limits can now be derived as follows.

Upper bounof $S_p = \lim_{w \to 0} S_p = p\text{-}1$ (10)

Lowerbound of $S_p = \lim_{w \to \infty} S_p = 1$ (11)

From the above analysis, it is apparent that the ratio w is critical to the speedup of the parallel algorithm. In the proposed algorithm, since Step 4 is the most computationally intensive part, which is required to solve n linear programming models, the ratio w is therefore very small and the speedup should be approximate to the upper bound limit. That is, the speedup would approach n when there are enough processors in the DCE.

4. Numerical experiments

In this section we present the numerical experiments results in order to justify the performance of the proposed algorithm. The algorithm presented was implemented in a cluster of distributed network workstations, which were connected via an optical fiber link. We used the Parallel Virtual Machine (PVM) library routines to develop the message-passing environment for distributed computing. PVM enables a collection of heterogeneous computer systems to be viewed as a single parallel virtual machine and has been widely adopted by researchers [13]. The algorithm code was programmed in FORTRAN/77.

The method for numerical experiments in this study was similar to the one proposed by Pisinger [8], where the number of model constraints ranged from 20 to 50 to 120. For each problem type, five sets of models, generated using different random-number generator seeds, were solved. The average CPU time (five replications for each instance) required to solve each type of test problem with respect to the different numbers of processors used during experiments is plotted in Figure 1. The speedup earned in the numerical experiments as well as the mean speedup of the proposed algorithm are calculated and plotted in Figure 2.

(Insert Figure 1 and Figure 2 about here)

It is clear that the numerical performance of the proposed algorithm is impressive based on the experimental results demonstrated in Figure 1. When solving a modest problem sized problem (120 constraints), the CPU time saved in a DCE could be more than six fold. Given that a distributed computing environment is very common in many computer centers and PVM is a shareware, the presented algorithm is useful to many researchers in solving the linear programming problems.

The best speedup obtained in the numerical experiments was 6.36 for the model with 120 constraints. In general, the speedups earned in the experiments were close to the mean speedups as expected (Figure 2). It is also interesting to note that near linear speedup was achieved, which means that the proposed algorithm can take full advantage of the distributed computing power as the size of the problem increases.

5. Conclusions

Distributed computing on clusters of workstations is attractive and cost-effective to researchers for evolving processing and networking technologies. This study developed a parallel linear programming algorithm and evaluated its performance on a DCE. The numerical results show that the speedup of the proposed algorithm approaches linearity, which is consist with its analytical performance. We conclude that the presented algorithm is efficient and becomes a useful reference solution model for LP applications, especially for large-scale problems.

The proposed algorithm was implemented on a PVM system (a portable distributed computing environment). This software is free to the public and has been installed on many networked computing platforms. We are currently working on porting our computer codes into other computing environments and testing the algorithm on a wider variety of test problems. It is safe to state that further study on the development of some mathematical programming algorithms that could also take advantage of distributed computing power requires greater research efforts.

Reference

1. Forrest J. J. H., Tomlin J. A. implementing the simplex method for the optimization subroutine library, IBM Systems Journal 1992; 31(1):11-25.
2. Lyu J., Gunasekaran A., Kachitvichyanukul V. Towards a portable and efficient environment for parallel computing, International Journal of Systems Science 1995; 26(6): 1333-1341.
3. Ashford R. W., Connard P., Daniel R. Experiments in solving mixed integer programming problems on small array of transputers, Journal

of the Operational Research Society 1992; 43(5): 61-72.

4. Viswanathkumar G., Srinivasan G. A bound and bound algorithm to minimize completion time variance on a single processor 2003; 30: 1135-1150.
5. Romeijn H. E., Smith R. L. Parallel algorithms for solving aggregated shortest-path problems, Computers and Operations Research 1999; 26: 941-953.
6. Hiller F. S., Lieberman G. J., Lieberman G. Introduction to Operations Research 1995, New York: McGraw-Hill.
7. Dantzig G. B., Wolfe P. The decomposition principle for linear programs, Operations Research 1960; 8: 101-111.
8. Chinneck J. W. Computer codes for the analysis of infeasible linear programs, Journal of the Operational Research Society 1996; 47: 61-72.
9. Martinson R. K., Tind J. An interior point method in Dantzig-Wolfe decomposition, Computers and Operations Research 1999; 26: 1195-1216.
10. Sunderam V. S. Heterogeneous network computing: the next generation, Parallel Computing 1997; **23**: 121-135.
11. Sekharan C. N., Goel V., Sridhar R. Load balancing methods for ray tracing and binary tree computing using PVM, Parallel Computing1995; 21: 1963-1978.
12. Quinn M. J. Parallel Computing: Theory and Practice 1994, McGraw Hill, N.Y.
13. Sunderam V. S., Geist G. A., Dongarra J., Manchek R. The PVM concurrent computing system: evolution, experiences, and trends, Parallel Computing 1994; 20: 531-545.
14. Pisinger D. An expanding-core algorithm for the exact 0-1 Knapsack problem, European Journal of Operational Research 1995; 87: 175-187.

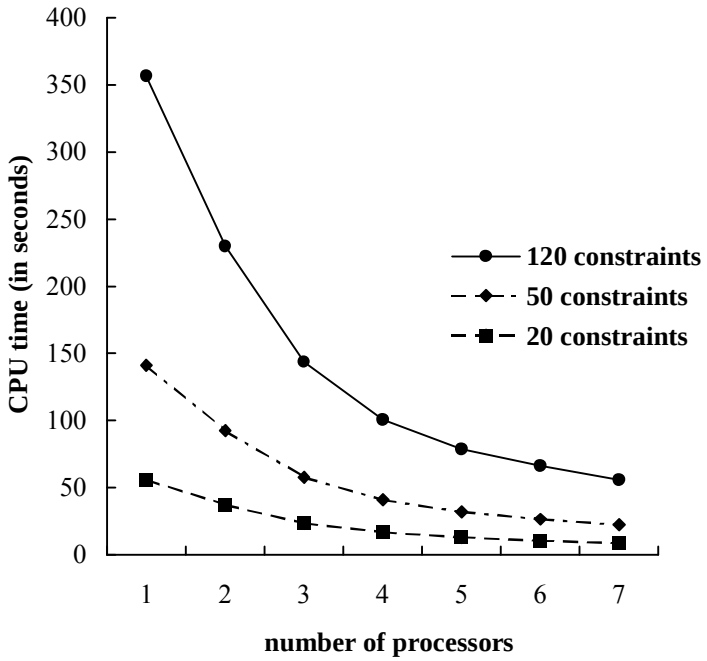

Figure 1. CPU time (in seconds) versus number of processors

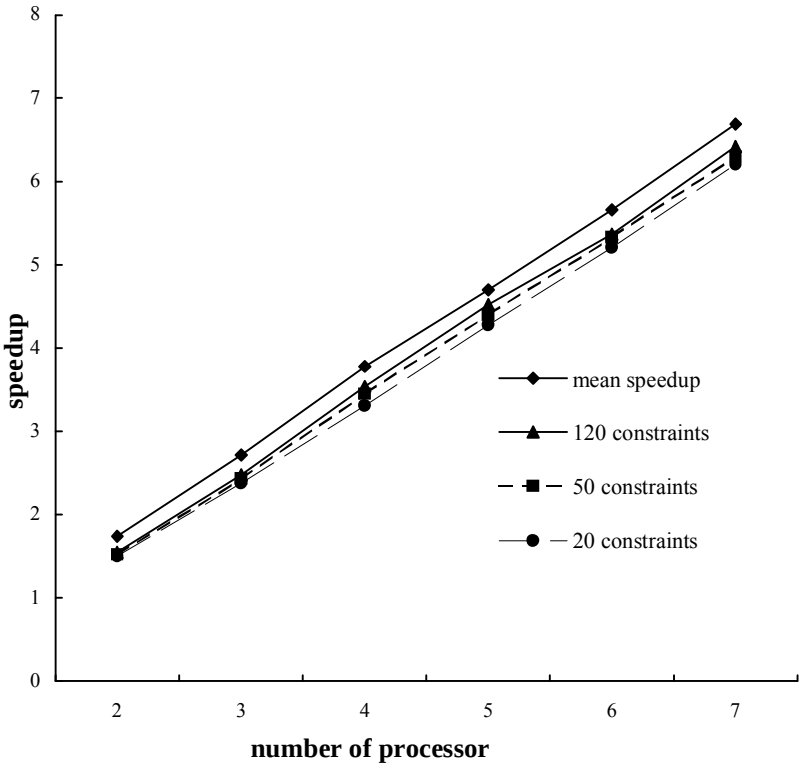

Figure 2. Mean speedup and speedup versus number of processors